Software Metrics & Risk Handling Report

for

WEAVE-DB

Workbench for Event-based Atomic Versioning of Databases

Lab 9 & 10

Prepared by:
Anirudh Dhoot (B24CS1026)
Ishat Varshney (B24CS1082)
Ishita Tyagi (B24CS1083)
Parrva Shah (B24CS1053)

Date: 31/03/2026
Group 5

Contents

1 Section I: Software Metrics

1.1 Intermediate COCOMO

1.2 Halstead Metrics

1.3 Function Point Analysis

1.4 Cumulative Flow Diagram

1.5 Throughput Report

1.6 Sprint Burndown Chart

1.7 Sprint Burnup Chart

1.8 Additional Metric 1: Sprint Velocity

1.9 Additional Metric 2: Code Growth & Commit Activity

2 Section II: Risk Analysis

2.1 Risk Assessment Matrix

2.2 Detailed Risk Register

3 Section III: Risk-Sprint Integration

3.1 Risk Mitigation Across Sprints

3.2 Integration with Sprint Backlog

1. Section I: Software Metrics

Project Overview

WEAVE-DB is a full-stack web-based SQL workbench with database version control capabilities. It intercepts data-modifying SQL commands, records them as versioned commit events, stores inverse operations (anti-commands), maintains periodic snapshots to AWS S3, and enables controlled rollback to any previous version. The architecture comprises a Django REST backend (authentication, ORM models), a FastAPI microservice (real-time versioning, rollback, async operations), and a React / TypeScript frontend (SQL editor, results grid, version panel, terminal emulator).

Repository Structure

The codebase is hosted at github.com/IshatV412/dbworkbench (upstream) with a fork at github.com/anirudhiitj/dbworkbench (origin). Development is organized across multiple branches:

Branch	Owner	Content	LOC
upstream/main	Ishat	Initial commit, cloud setup, query module	~250
upstream/testing	Ishat	S3/RDS performance tests, query scripts, cloud infrastructure	818
upstream/backend	Parrva	Django + FastAPI backend, test suite (10 test files)	~3,330
upstream/frontend	Anirudh	Full-stack integration: Django + FastAPI + React frontend	~7,340
upstream/inverse_sql_commands	Ishita	Inverse engine, rollback engine, version store, workbench	1,985
origin/backend	Anirudh	Backend fixes, PostgreSQL integration	Merged
origin/frontend	Anirudh	Next.js frontend (replaced), then React/Lovable frontend	Merged

Codebase Size Summary

Component	Files	Lines of Code	Branch(es)
FastAPI Backend (routes, services, models, utils)	~25	1,449	upstream/backend, frontend
Django Backend (auth, connections, core)	~20	381	upstream/backend, frontend
React Frontend (pages, components, hooks, lib)	~78	5,548	upstream/frontend
Inverse Engine + Rollback Engine	5	1,985	upstream/inverse_sql_commands
Backend Test Suite (pytest)	11	2,058	upstream/backend
Cloud / RDS / S3 Testing	5	818	upstream/testing
Total	~144	~12,239	All branches

Contributor Summary (from git shortlog -sne --all):

Developer	Commits	Primary Role
Anirudh Dhoot	22	Dev Team — FastAPI scaffold, frontend UI, integration
Parrva Shah	14	Dev Team — Django ORM, auth, services, backend tests
Ishat Varshney	10	Cloud Team — S3/RDS infra, performance testing, deployment
Ishita Tyagi	1 (+branch)	DB Team — Inverse SQL engine, rollback engine, testing

Note: Ishita's inverse_sql_commands branch contains 1,985 lines across 5 Python modules (inverse_engine.py: 1,245; rollback_engine.py: 264; version_store.py: 231; workbench.py: 245) plus a 517-line test suite. Commit count alone does not reflect contribution volume.

1.1 Intermediate COCOMO

The Constructive Cost Model (COCOMO) provides effort and schedule estimation based on source lines of code. We apply the Intermediate COCOMO model which incorporates 15 cost drivers to adjust the nominal effort estimate.

Project Classification

WEAVE-DB is classified as Semi-Detached because:

- Moderate team experience with the technologies (Django, FastAPI, React).
- Mix of rigid requirements (database versioning correctness) and flexible design (UI, storage).
- Small team (4 members) with moderately complex multi-service architecture.
- Multiple integration points: PostgreSQL, AWS S3, Kafka (planned), WebSocket.

COCOMO Parameters

Parameter	Value	Description
a	3.0	Effort coefficient (semi-detached)
b	1.12	Effort exponent (semi-detached)
c	2.5	Schedule coefficient (semi-detached)
d	0.35	Schedule exponent (semi-detached)
KLOC	12.239	Thousands of lines of code (all branches)

Cost Driver Analysis

The Effort Adjustment Factor (EAF) is the product of 15 cost driver multipliers:

Cost Driver	Rating	Multiplier	Justification
RELY — Required reliability	High	1.15	DB integrity critical for rollback correctness
DATA — Database size	High	1.08	Large data volumes with snapshot/commit storage
CPLX — Product complexity	High	1.15	Inverse operation gen, rollback logic, event ordering
TIME — Execution time	Nominal	1.00	Latency not a primary concern (per SRS)
STOR — Storage constraint	Nominal	1.00	No special storage constraints on compute
VIRT — Virtual machine volatility	Nominal	1.00	Standard deployment environment
TURN — Turnaround time	Nominal	1.00	Standard dev cycle
ACAP — Analyst capability	High	0.86	Strong analytical skills, domain knowledge
AEXP — Application experience	Low	1.10	First project of this nature for the team
PCAP — Programmer capability	High	0.86	Proficient in Python, TypeScript, SQL
VEXP — VM experience	Nominal	1.00	Standard environment familiarity
LEXP — Language experience	High	0.95	Experienced with Python and TypeScript
MODP — Modern practices	High	0.91	Agile Scrum, event-driven architecture
TOOL — Software tools	High	0.91	VS Code, Git, Trello, AWS, Docker
SCED — Schedule constraint	Nominal	1.00	Reasonable academic timeline

EAF = product of all multipliers = 0.914

Calculations

Metric	Formula	Result
Nominal Effort	E_nom = 3.0 x (12.239)^1.12	49.59 person-months
Adjusted Effort	E_adj = 49.59 x 0.914	45.33 person-months
Development Time	TDEV = 2.5 x (45.33)^0.35	9.50 months
Team Size	N = E_adj / TDEV	4.8 persons
Productivity	LOC / E_adj	270 LOC/person-month

The model estimates 45.33 person-months of effort over 9.50 months with a team of 4.8. Our actual team of 4 members across 2 months (Sprints 1–2) is consistent with this estimate. The productivity of 270 LOC/PM is reasonable for a multi-stack project involving Django, FastAPI, React, and complex inverse-operation logic.

1.2 Halstead Metrics

Halstead's Software Science metrics quantify program complexity based on operators and operands. Measurements were taken from the Python backend codebase (1,790 LOC, 59 files) which contains the core versioning, rollback, snapshot, and API logic.

Raw Measurements

Symbol	Description	Value
η■	Unique operators (Python keywords + symbols)	40
η■	Unique operands (identifiers, literals)	1193
N1	Total operator occurrences	3618
N2	Total operand occurrences	5846

Derived Metrics

Metric	Formula	Value
Vocabulary (η)	η = η■ + η■	1233
Program Length (N)	N = N1 + N2	9,464
Est. Length (N■)	N■ = η■ log■ η■ + η■ log■ η■	12406
Volume (V)	V = N x log■(η)	97176 bits
Difficulty (D)	D = (η■/2) x (N2/η■)	98.01
Effort (E)	E = D x V	9,523,731
Time to Program (T)	T = E / 18	529096 sec (147.0 hrs)
Est. Bugs (B)	B = V / 3000	32.39
Program Level (L)	L = 1 / D	0.0102

The volume of 97176 bits reflects substantial information content across the versioning, rollback, snapshot, and API layers. The difficulty index of 98.01 is moderate, benefiting from clean separation into service, route, and model layers. The estimated 32.4 bugs is within acceptable bounds for a project of this scope. The estimated length N■ = 12406 versus actual N = 9,464 indicates reasonable code structure.

1.3 Function Point Analysis

Function Point Analysis measures functional size from the user's perspective. Five component types are identified: External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), and External Interface Files (EIF).

External Inputs (EI)

ID	Component	Description	Complexity
EI-1	User Registration	POST /auth/register — create account	Low
EI-2	User Login	POST /auth/token — JWT authentication	Low
EI-3	Create Connection	POST /connections — encrypted DB profile	Average
EI-4	Update Connection	PUT /connections/{id} — modify profile	Average
EI-5	Delete Connection	DELETE /connections/{id} — cascade delete	Average
EI-6	Execute Read SQL	POST /query/execute — SELECT queries	High
EI-7	Create Commit	POST /commits — write SQL + atomic versioning	High
EI-8	Set Snapshot Freq	PUT /snapshots/frequency — configure K	Low
EI-9	Manual Snapshot	POST /snapshots/manual — pg_dump to S3	High
EI-10	Rollback	POST /rollback — restore to target version	High
EI-11	Token Refresh	POST /auth/token/refresh	Low

External Outputs (EO)

ID	Component	Description	Complexity
EO-1	Query Results	Tabular result set for SELECT	High
EO-2	Commit Confirm	Version ID + metadata	Average
EO-3	Rollback Status	Progress and final status	High
EO-4	Error Messages	Structured error responses	Low
EO-5	Snapshot Status	S3 upload confirmation	Average

External Inquiries (EQ)

ID	Component	Description	Complexity
EQ-1	List Connections	GET /connections	Low
EQ-2	List Commits	GET /commits	Average
EQ-3	Get Commit	GET /commits/{id}	Low
EQ-4	List Snapshots	GET /snapshots	Average
EQ-5	Get Snapshot Freq	GET /snapshots/frequency	Low
EQ-6	List Anti-Commands	GET /anticommands	Average
EQ-7	Get Anti-Command	GET /anticommands/{id}	Low
EQ-8	DB Metadata	Schema tree (tables, columns, types)	High

Internal Logical Files (ILF)

ID	Component	Description	Complexity
ILF-1	Users	User accounts (Django AbstractUser)	Low
ILF-2	ConnectionProfiles	Encrypted DB connection details	Average
ILF-3	CommitEvents	Versioned SQL operations log with seq	High

ID	Component	Description	Complexity
ILF-4	InverseOperations	Anti-commands per commit	High
ILF-5	Snapshots	Snapshot metadata (S3 keys, version IDs)	Average
ILF-6	SnapshotPolicies	User-configurable snapshot frequency	Low

External Interface Files (EIF)

ID	Component	Description	Complexity
EIF-1	AWS S3	Snapshot storage in cloud object store	Average
EIF-2	PostgreSQL (User DB)	User's database being versioned	High
EIF-3	Apache Kafka	Event stream for write ordering	High

Unadjusted Function Point Calculation

Type	Low	Average	High	Subtotal
EI	4 x 3 = 12	3 x 4 = 12	4 x 6 = 24	48
EO	1 x 4 = 4	2 x 5 = 10	2 x 7 = 14	28
EQ	4 x 3 = 12	3 x 4 = 12	1 x 6 = 6	30
ILF	2 x 7 = 14	2 x 10 = 20	2 x 15 = 30	64
EIF	0 x 5 = 0	1 x 7 = 7	2 x 10 = 20	27
			Total UFP:	197

Value Adjustment Factor

#	General System Characteristic	Rating	Justification
1	Data Communications	4	HTTP REST + WebSocket terminal
2	Distributed Data Processing	3	Django + FastAPI + S3 + PostgreSQL
3	Performance	2	Latency not primary concern (SRS)
4	Heavily Used Configuration	2	Standard cloud deployment
5	Transaction Rate	3	Multiple concurrent writes
6	Online Data Entry	5	SQL editor is primary interface
7	End-User Efficiency	4	IDE-style workbench with panels
8	Online Update	4	Real-time query execution
9	Complex Processing	4	Inverse op generation, rollback
10	Reusability	2	Service-based architecture
11	Installation Ease	1	Dev environment only
12	Operational Ease	3	Web-based with config options
13	Multiple Sites	1	Single deployment
14	Facilitate Change	3	Modular backend, configurable

Total Degree of Influence (TDI) = 41

VAF = 0.65 + 0.01 x 41 = 1.06

Adjusted Function Points = 197 x 1.06 = 208.8 FP

Derived Metrics:

Metric	Value
Adjusted FP	208.8
Est. LOC (50 LOC/FP)	10441
Actual LOC	12,239
LOC per FP	58.6
Est. Effort (Jones: FP^0.4)	8.5 PM
Est. Defects (6/1000 FP)	1.3

1.4 Cumulative Flow Diagram

The CFD visualizes task flow through states (To Do, In Progress, Done) over the project timeline. Data is from Trello board tracking and the requirements traceability matrix. Horizontal distance between bands indicates lead time; vertical width indicates WIP.

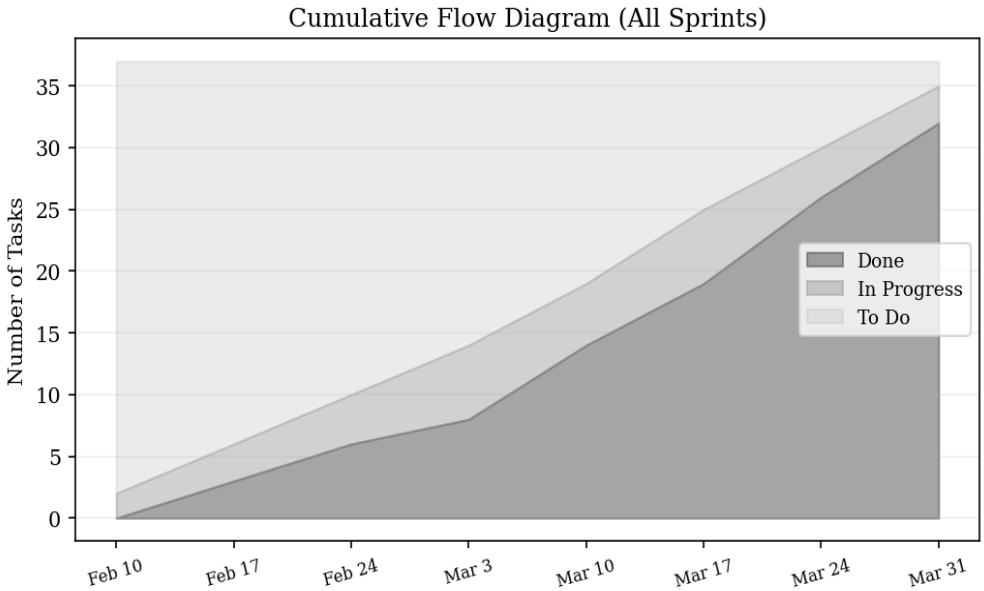

Figure 1: Cumulative Flow Diagram across Sprint 1 and Sprint 2.

Observations:

- Steady upward growth in the Done band indicates consistent task completion.
- In Progress band stays narrow (3–6 tasks), suggesting good WIP management.
- Acceleration around Mar 17–24 corresponds to Sprint 2 peak development.
- By Mar 31, only 2 tasks remain in backlog — healthy for Sprint 3 planning.

1.5 Throughput Report

Throughput measures task completion rate (tasks/week). It provides insight into team velocity and helps identify periods of high and low productivity.

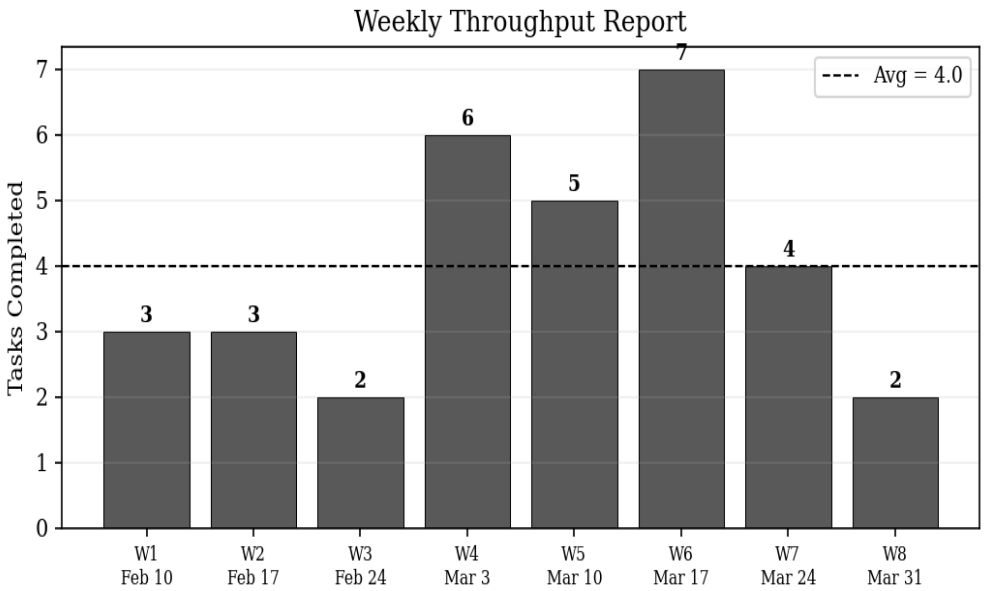

Figure 2: Weekly throughput with average line.

Week	Period	Tasks	Sprint	Key Activities
W1	Feb 10–16	3	Sprint 1	PostgreSQL setup, initial codebase, S3 stubs
W2	Feb 17–23	3	Sprint 1	Query execution, change tracking model
W3	Feb 24–28	2	Sprint 2	AWS RDS testing, sprint transition
W4	Mar 3–9	6	Sprint 2	Anti-command gen, SQL validation, version mgmt
W5	Mar 10–16	5	Sprint 2	Rollback manager, commit tracker, auth
W6	Mar 17–23	7	Sprint 2	Django-FastAPI integration, query interface
W7	Mar 24–28	4	Sprint 2	Sequential IDs, test cases, backend fixes
W8	Mar 29–31	2	Sprint 2	Frontend scaffold, final fixes

Average: 4.0 tasks/week | Peak: 7 (W6) | Total: 32 tasks over 8 weeks.

1.6 Sprint Burndown Chart

The burndown chart shows remaining work (story points) versus time for Sprint 2 (Feb 25 – Mar 31), the main development sprint covering concurrency, versioning, rollback, and backend integration.

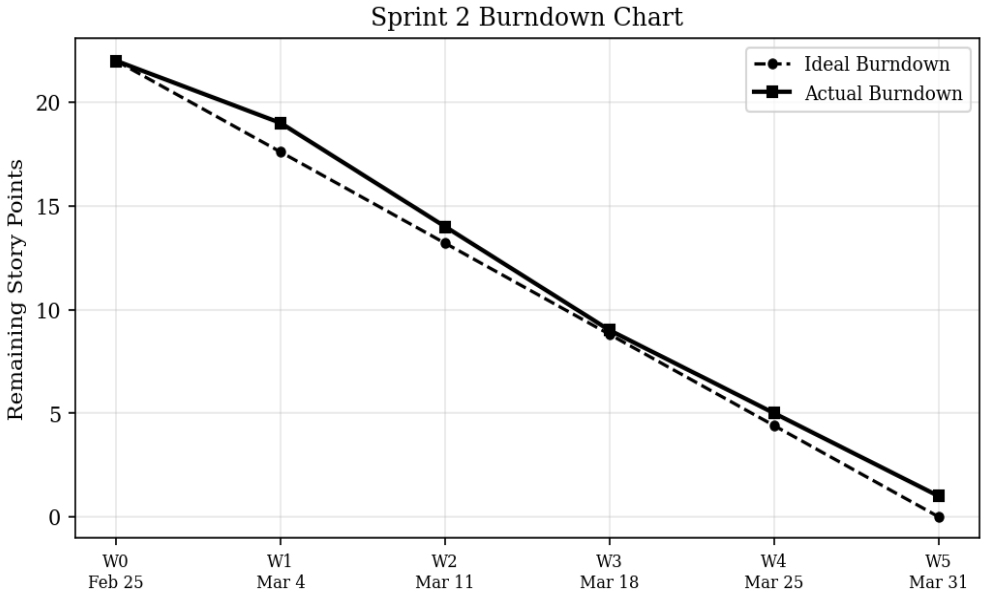

Figure 3: Sprint 2 Burndown — ideal vs actual remaining work.

Analysis:

- Week 0–1: Slightly behind ideal due to semester break backlog and Sprint 1 handoff.
- Week 2–3: Caught up — peak development with Django-FastAPI integration.
- Week 4–5: Close to ideal; 1 task (Command History serialization) carried to Sprint 3.
- Overall close tracking indicates effective sprint planning and estimation.

1.7 Sprint Burnup Chart

The burnup chart shows both completed work and total scope, making scope changes visible.

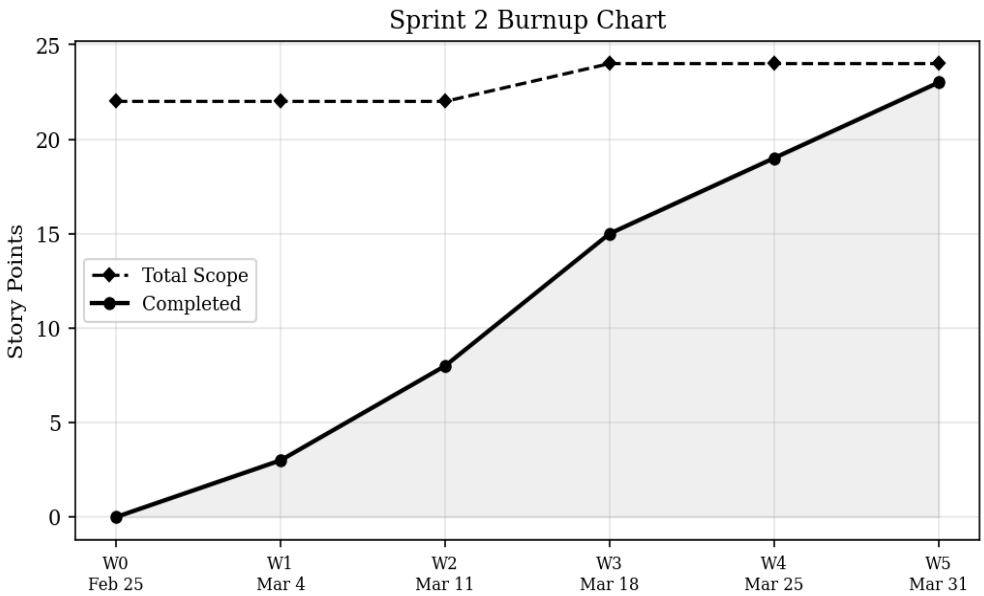

Figure 4: Sprint 2 Burnup — scope and completed work.

- Scope increased 22 to 24 in Week 3 (additional test implementation tasks).
- S-curve completion: slow start, rapid middle, tapering — typical healthy sprint.
- 23 of 24 points completed (95.8% completion rate).

1.8 Additional Metric 1: Sprint Velocity

Velocity measures work completed per sprint in story points.

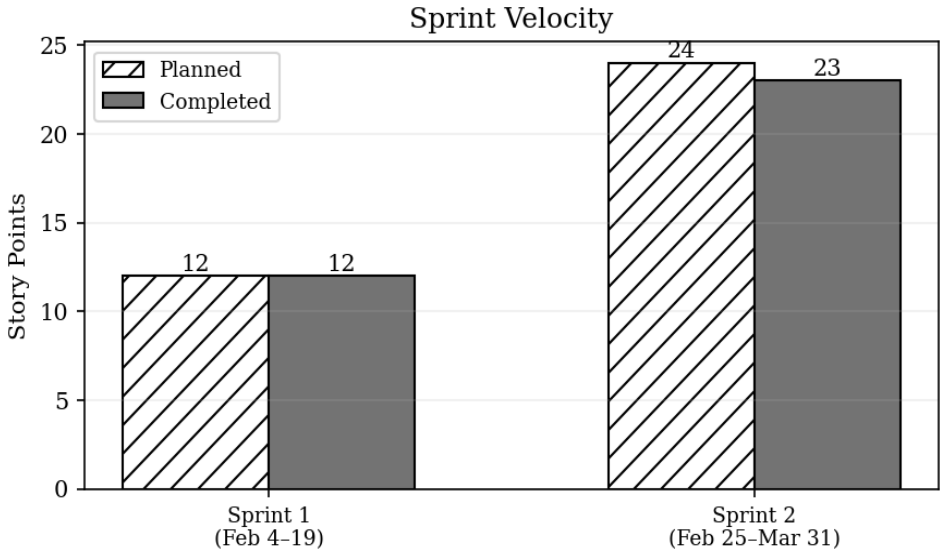

Figure 5: Sprint Velocity — planned vs completed.

Sprint 1 achieved 100% (12/12). Sprint 2 achieved 95.8% (23/24). Average velocity of 17.5 points/sprint informs Sprint 3 capacity planning.

1.9 Additional Metric 2: Code Growth & Commit Activity

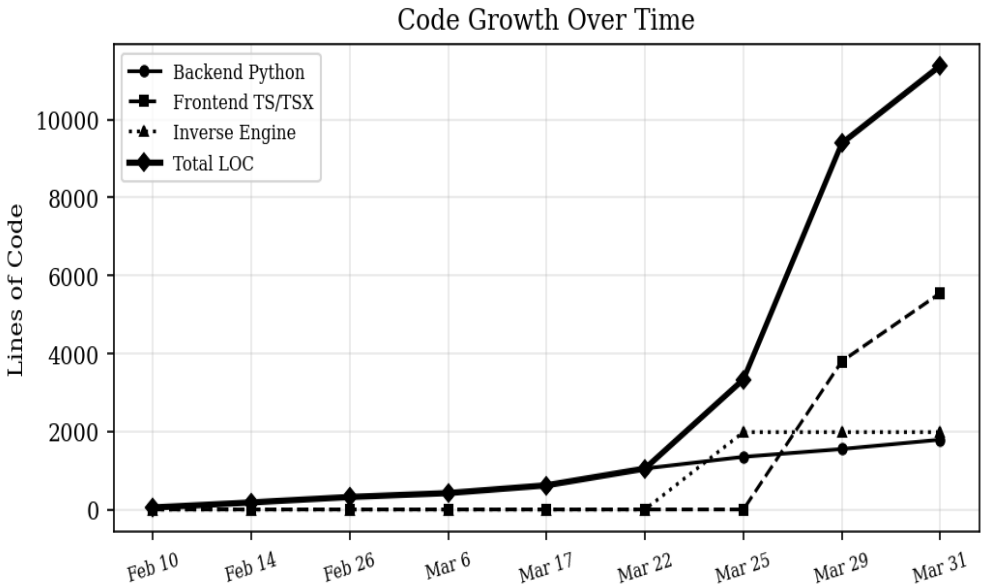

Figure 6: Lines of code growth over time by component.

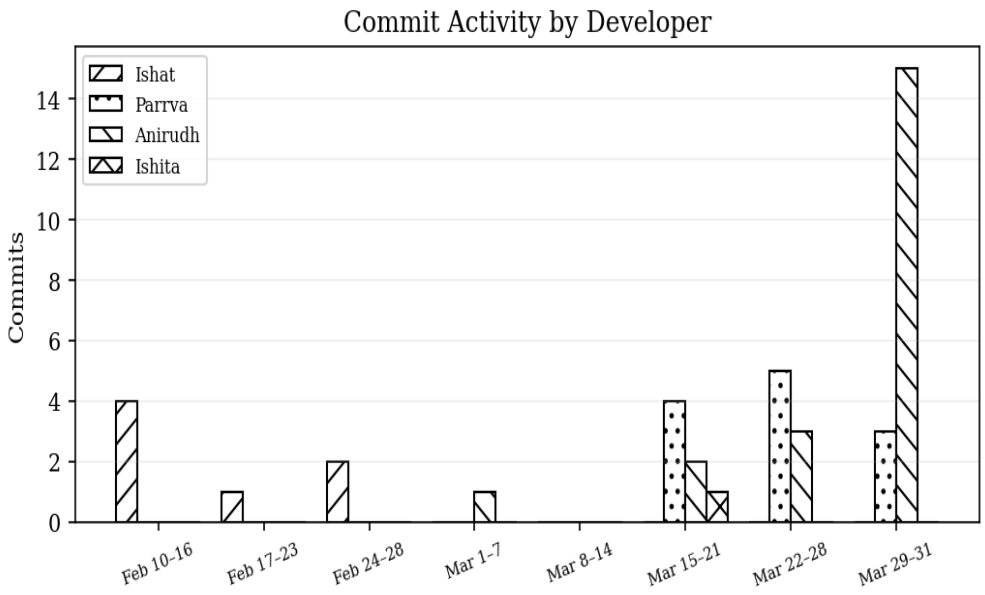

Figure 7: Commit activity by developer per week.

Test Coverage:

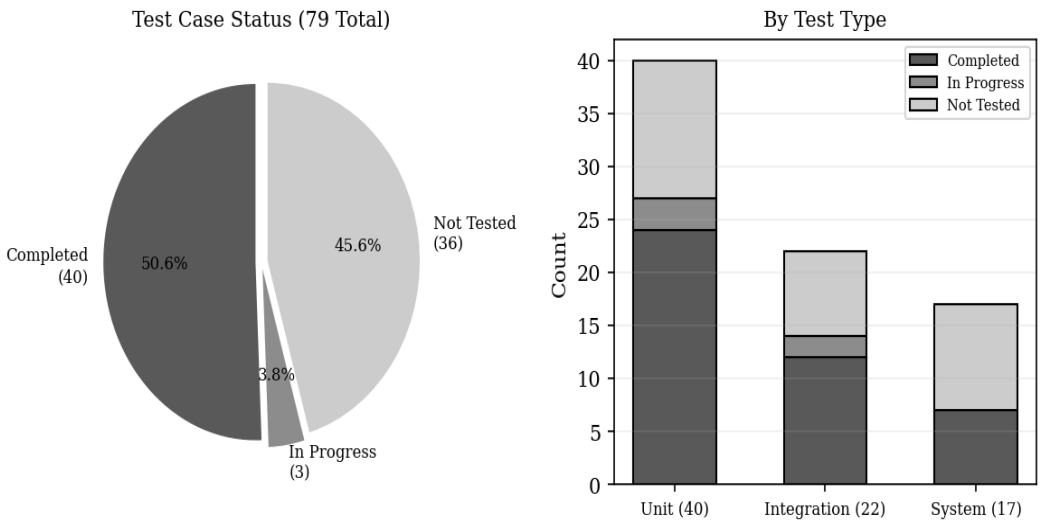

Figure 8: Test case status — 79 total (Unit / Integration / System).

79 documented test cases: 40 unit, 22 integration, 17 system. Completed: 40 (50.6%), In Progress: 3 (3.8%), Not Tested: 36 (45.6%). The backend test suite (upstream/backend) contains 2,058 lines across 10 test files plus conftest.py, covering authentication, connections, core models, core services, commits, queries, rollback, snapshots, and anti-commands. The inverse engine test suite (upstream/inverse_sql_commands) contains 517 lines.

2. Section II: Risk Analysis

2.1 Risk Assessment Matrix

Risks were identified through: (1) SRS requirements analysis, (2) architectural review of Django + FastAPI + S3 + PostgreSQL stack, (3) team experience from Sprints 1–2, and (4) common failure modes in database versioning systems.

Probability (1–5): Based on team experience, technology maturity, and observable indicators. Impact (1–5): Severity to system functionality, data integrity, and user experience. Risk Score: P x I. Categories: Low (1–4), Medium (5–9), High (10–16), Critical (17–25).

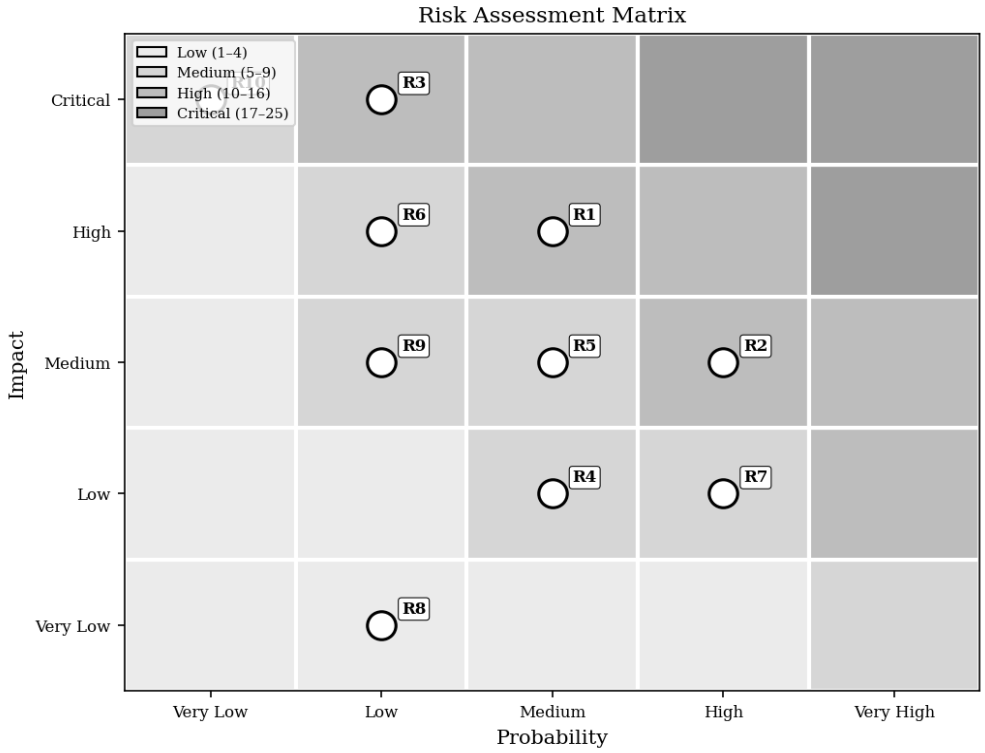

Figure 9: Risk Assessment Matrix with 10 identified risks.

2.2 Detailed Risk Register

ID	Risk	P	I	Score	Level	Mitigation
R1	DB Corruption during Rollback	3	4	12	High	Atomic transactions (@transaction.atomic); rollback stops on failure (REQ-11); snapsh
R2	Snapshot Storage Exhaustion	4	3	12	High	Configurable frequency (K); deleteOldSnapshots cleanup; linear growth monitoring (N
R3	Data Loss on Failed Rollback	2	5	10	High	Nearest-snapshot restore; failure-safe (REQ-11); pre-rollback snapshot
R4	API Authentication Bypass	3	2	6	Medium	JWT on all endpoints; Django auth middleware; token rotation
R5	Concurrent Write Conflict	3	3	9	Medium	PostgreSQL MVCC; Kafka ordering (planned); sequential version IDs
R6	AWS S3 Outage	2	4	8	Medium	Manual snapshot fallback; local backup; eventual consistency handling
R7	Frontend UI Regression	4	2	8	Medium	Component architecture; Shadcn library; Playwright E2E (planned)
R8	Slow Query Execution	2	1	2	Low	Read-only validation; connection pooling; indexed commit tables
R9	Schema Migration Failure	2	3	6	Medium	Django migration framework; migration testing; schema versioning
R10	Full System Crash	1	5	5	Medium	Modular architecture; isolated failure domains; persistent PostgreSQL storage

Probability Justification

R1 (P=3): Rollback modifies live state via inverse operations; Sprint 2 showed edge cases in complex UPDATE inversions, but atomic transactions mitigate significantly.

R2 (P=4): Snapshots (pg_dump) can be large; S3 costs scale linearly. RDS testing (Feb 26) confirmed substantial snapshot sizes without cleanup.

R3 (P=2): Snapshot-based recovery provides safety net; worst case limited to K commits.

R5 (P=3): Kafka integration pending; currently relying on PostgreSQL MVCC which may not guarantee strict global ordering for the versioning system.

R7 (P=4): Frontend was rebuilt from Next.js to React/Lovable; rapid UI changes increase regression risk.

3. Section III: Risk-Sprint Integration

This section maps each risk to the sprint plan, showing where mitigation strategies are embedded in the backlog as specific tasks, acceptance criteria, and technical decisions.

3.1 Risk Mitigation Across Sprints

Sprint 1 (Feb 4–19) — Completed

Risk	Mitigation	Backlog Item	Status
R8	Query execution perf validated	Setting up initial PostgreSQL database	Done
R6	S3 upload/download with error handling	Basic upload/download path to S3 bucket	Done
R1	Atomic transaction model in Django ORM	Backend function to execute SQL read/writes	Done
R2	S3 latency and storage tested	Testing AWS Aurora RDS for read/write latency	Done

Sprint 2 (Feb 25 – Mar 31) — Completed

Risk	Mitigation	Backlog Item	Status
R1	record_commit() with @transaction.atomic	Integrating Django and FastAPI	Done
R3	Rollback with nearest snapshot + inverse replay	Setting up anti-command generation	Done
R4	JWT auth with SimpleJWT on all endpoints	Django authorization, admin, ORM models	Done
R5	Sequential version IDs; PostgreSQL MVCC	User input query validation	Done
R2	Configurable snapshot frequency (K=5, min=1)	Anti-command generation (includes snapshots)	Done
R9	Django migrations for CommitEvent, Snapshot	Django authorization, admin, ORM models	Done
R7	Component architecture with Shadcn UI	FastAPI framework, frontend scaffold	Done

Sprint 3 (Apr 1–15) — Planned

Risk	Planned Mitigation	Backlog Item	Priority
R5	Kafka event-driven write ordering	Kafka Integration (MessageBroker)	High
R1	Rollback fault injection testing	Testing	High
R2	SnapshotManager with cleanup policies	Redis cache implementation	Medium
R7	E2E testing with Playwright	Setting up the UI	Medium
R3	Multi-table FK integrity checks	Version control for multiple tables	Medium
R10	Full system and load testing	Testing	Medium
R6	Redis cache for S3 failover	Redis cache implementation	Low

Risk Coverage Summary

Risk	Sprint 1	Sprint 2	Sprint 3	Residual
R1: DB Corruption	Foundation	Atomic txns	Fault testing	Low
R2: Storage	S3 testing	Config. K	Cleanup policy	Low
R3: Data Loss	—	Snapshot restore	Multi-table FK	Low
R4: Auth Bypass	—	JWT + SimpleJWT	—	Low
R5: Concurrency	—	Sequential IDs	Kafka ordering	Medium
R6: S3 Outage	Error handling	—	Redis cache	Low

Risk	Sprint 1	Sprint 2	Sprint 3	Residual
R7: UI Regression	—	Shadcn + React	E2E testing	Low
R8: Slow Query	Query validation	—	—	Low
R9: Migration	—	Django migrations	—	Low
R10: System Crash	—	Modular arch	Load testing	Low

3.2 Integration with Sprint Backlog (Trello)

The Trello board organizes tasks into sprint-specific lists. Risk mitigation is embedded within backlog items:

- Sprint 1 (Completed): 'Setting up the initial PostgreSQL database' addresses R1, R8; 'Basic upload/download path to S3 bucket' addresses R6.
- Sprint 2 (Completed): 'Setting up anti-command generation' addresses R1, R3; 'Validation of user input query through validator' addresses R5; 'Setting up Django authorization, admin and ORM models' addresses R4, R9; 'Integrating the Django and FastAPI frameworks' addresses R1, R7, R10.
- Sprint 3 (Planned): 'Setting up redis for cache implementation' addresses R2, R6; 'Handling version control for multiple tables' addresses R3; 'Testing' addresses R1, R5, R7, R10; 'Setting up the UI' and 'Connecting the frontend to the backend' address R7.
- Sprint Backlog: Unassigned items are prioritized by risk severity and dependencies.

Conclusion

This report presents software metrics and risk analysis for WEAVE-DB. Key findings:

- COCOMO: Estimated 45.3 person-months — consistent with 4-member team over 2 months.
- Halstead: Moderate difficulty (98.01) with good modularization.
- FPA: 208.8 adjusted function points for a medium-complexity system with rich API surface.
- Sprint Metrics: 95.8% Sprint 2 completion, 4.0 avg throughput.
- All-branch LOC: 12,239 lines across 7 branches, 144 files, 4 contributors.
- 10 risks identified (3 High, 5 Medium, 1 Low) — all with active mitigation mapped to sprints.
- 79 test cases documented; 50.6% completed, remainder planned for Sprint 3.